

5장 의미 분석 (타입 검사)

최광훈

전남대학교

목차

의미 분석(Semantic analysis)

[실습] MiniJava 타입 검사

의미 분석(타입 검사)

컴파일 과정은 파서가 발견한 프로그램의 구문 구조에 의해 주도됨

의미 루틴은

- 구문 구조에 따라 프로그램의 의미를 해석
- 두 가지 목적
 - 1) 문맥에 의존하는 정보를 도출하여 분석을 마무리하고,
 - 2) IR 또는 목적 코드 생성을 시작하는 것
- 문맥 자유 문법의 각 생성 규칙 또는 구문 트리의 하위 트리 단위로 진행

문맥 의존 분석

문맥 자유 문법만으로는 답할 수 없는 질문들

- x 는 스칼라인가, 배열인가, 아니면 함수인가?
- x 는 사용되기 전에 선언되었는가?
- 선언되었지만 사용되지 않은 이름이 있는가?
- 이 참조가 가리키는 것은 어느 선언인가?
- 이 식은 타입에서 일관적으로 구성되어 있는가?
- 참조의 차원이 선언과 일치하는가?
- x 는 어디에 저장될 수 있는가? (예: 힙, 스택, ...)

문맥 의존 분석

문맥 자유 문법만으로는 답할 수 없는 질문들

...

- `p`는 `malloc()`의 결과를 참조하는가?
- `x`는 사용되기 전에 정의되었는가?
- 배열 참조가 범위 내에 있는가?
- 함수 `f`는 상수 값을 생성하는가?
- `f`는 메모 함수로 구현될 수 있는가?

문맥 의존 분석

문맥에 의존하는 분석은 왜 어려운가?

- 답하기 위해서 구문이 아니라 값이 필요하기 때문
- 질문과 답이 지역에 있지 않은 정보(non-local info.)가 필요하기 때문
- 답을 구하기 위해 계산이 필요하기 때문

문맥 의존 분석

여러가지 대안

- **AST 속성 문법 (attribute grammars)**: 비지역적 계산을 기술하고 자동 계산
- **심볼 테이블(symbol table)**: 컴파일러가 분석한 사실들을 저장하는 자료 구조
 - : 변수 이름, 타입, 이름의 범위(**scope**), 메모리 위치(힙, 스택), 선언된 함수 타입
- 언어 설계: 애초에 언어를 단순하게 설계해서 문맥 의존하는 문제를 줄이거나 피하기

=> 심볼 테이블

심볼 테이블

심볼 테이블

- 어휘 이름(심볼)을 그에 대응하는 속성(타입, 내부 레이아웃, ...)과 연결

어떤 항목(심볼)을 넣어야 하는가?

- 변수 이름, 정의된 상수, 프로시저와 함수 이름, 리터럴(literal) 상수와 문자열
- 소스 코드의 라벨, 컴파일러가 생성하는 임시 변수

$(a + b * c) \Rightarrow (t1 = b * c; t2 = a + t1)$

심볼 테이블

구조체, 레코드, 클래스 같은 복합 타입의 내부 레이아웃 정보 관리

- 필드 이름과 타입, 필드의 오프셋(**offset**), 전체 크기(**length/size**)

- `struct Point { int x; int y; }`

`x`의 위치, `y`의 위치, 전체 크기

심볼 테이블 정보

- 이름, 타입, 차원 정보
- 프로시저 선언, 선언의 어휘 단계(**lexical level**)
- 저장 클래스
- 저장 위치에서의 오프셋
- 레코드라면, 구조 테이블에 대한 포인터
- 매개변수라면, 참조 전달(**by-reference**)인지 값 전달(**by-value**)인지
- 별칭(**alias**)이 가능한지? 가능하다면 어떤 다른 이름과?
- 함수 인자 개수와 각 인자의 타입

중첩 스코프를 위한 블록 구조 심볼 테이블

어떤 이름을 찾을 때, 그 이름의 가장 최근 선언을 찾아야 함

- 그 선언은 현재 스코프에 있을 수도 있고, 이를 감싸는 바깥 스코프에 있을 수도 있음
- 가장 안쪽 스코프(innermost scope)의 선언이 바깥 스코프의 선언보다 우선함

Key: 새로운 이름 선언은 대개 현재 스코프에만 추가됨

중첩 스코프를 위한 블록 구조 심볼 테이블

필요한 연산

- void put(Symbol key, Object value) : 이름을 해당 정보에 연결(binding)
- Object get(Symbol key) : 이름에 바인딩된 정보를 조회
- void beginScope() : 현재 심볼 테이블 상태를 따로 보관
- void endScope() : 가장 최근에 따로 보관한 심볼 테이블 상태를 복원

(디버깅 목적으로 지역 변수 목록을 유지하여, 현재 스코프의 변수들을 보여줄 수 있음)

속성 정보

속성(attributes)은 상수/변수/함수/클래스/... 선언의 내부 표현

심볼 테이블은 이름을 그에 대응하는 속성과 연결

이름은 그 의미에 따라 서로 다른 속성을 가질 수 있음

- 변수: 타입, 선언 수준, 프레임 내 위치(오프셋)
- 타입: 타입 정보, 크기, 정렬 단위
- 상수: 타입, 값
- 함수: 형식 매개변수(이름/타입), 반환 타입, 지역 선언 정보, 프레임 크기

심볼 테이블 구현

```
- package Symbol;

public class Symbol {
    public String toString();
    public static Symbol symbol(String s);
}

public class Table {
    public Table();
    public void put(Symbol key, Object value);
    public Object get(Symbol key);
    public void beginScope();
    public void endScope();
    public java.util.Enumeration keys();
}
```

put 함수는 기존 테이블을 수정하지 않고 새로운 테이블을 반환한다면,

새로운 버전의 테이블을 사용하는 동안에도 이전 버전의 테이블을 그대로 보관할 수 있으므로, **beginScope**와 **endScope**는 필요하지 않음.

심볼 테이블 구현

- 예) `int x = 1;`

`{`

`int y = 2;`

`int x = 3;`

`}`

`x_sym |-> 1`

`y_sym |-> 2`
`x_sym |-> 3`

`x_sym |-> 1`

push
by `beginScope()`

pop
by `endScope()`

심볼 구현

```
- package Symbol;
  public class Symbol {
      private String name;
      private Symbol(String n) {name=n; }
      private static java.util.Dictionary dict = new java.util.Hashtable();

      public String toString() {return name;}

      public static Symbol symbol(String n) {
          String u = n.intern();
          Symbol s = (Symbol)dict.get(u);
          if (s==null) {s = new Symbol(u); dict.put(u,s); }
          return s;
      }
  }
```

`intern()`은 같은 문자 시퀀스를 갖는 문자열에 대해 동일한 **String** 객체를 돌려주는 기능

```
String a = new String("abc").intern();
String b = new String("abc").intern();
```

a와 b는 같은 문자열 객체를 가리킴

심볼 구현

문자열(string)을 그대로 이름으로 쓰면 비효율적, 문자열을 심볼로 변환해서 사용

- 예) `abc := 10`

`abc + 1`

- 입력 프로그램 안에서는 문자열 "abc"가 여러 번 나타남. 이때 매번 문자열 "abc"를 직접 비교하면 비용이 큼

소스 코드 안의 문자열 이름 -> Symbol 모듈 -> 고유한 symbol 객체 -> environment, hash table, tree 등의 key로 사용

목차

의미 분석(Semantic analysis)

[실습] MiniJava 타입 검사

MiniJava 타입 검사

In chap5/handcrafted/Main.java

```
import syntaxtree.*;
import visitor.*;

public class Main {
    public static void main(String [] args) {
        for (String path : args) {
            try {
                java.io.InputStream in = new java.io.FileInputStream(path);
                MiniJavaParser parser = new MiniJavaParser(in);
                Program root = parser.Goal();
                // root.accept(new PrettyPrintVisitor());
                boolean success = new TypeCheckVisitor().check(root);
                if (success) {
                    System.out.println("Successfully type checked for " + path);
                } else {
                    System.err.println("Type check failed for " + path);
                }
            } catch (ParseException e) {
                System.err.println("Parse error in " + path + ":\n" + e.toString());
                System.exit(2);
            } catch (java.io.IOException e) {
```

MiniJava 타입 검사: 성공

In chap5/handcrafted/Main.java

- \$ java Main ../../programs/Factorial.java

Successfully type checked for ../../programs/Factorial.java

- 나머지 programs 예제들도 타입 검사 성공

: TreeVisitor.java를 타입 검사한 결과는?

MiniJava 타입 검사: 실패

In chap5/handcrafted/Main.java

- 10: int num_aux; 10번째 줄을 삭제

```
7  class Fac {
8
9      public int ComputeFac(int num){
10                 
11         if (num < 1)
12             num_aux = 1 ;
13         else
14             num_aux = num * (this.ComputeFac(num-1)) ;
15         return num_aux ;
16     }
17
18 }
```

타입 검사 에러가 발생한 곳의 라인과
컬럼 번호를 함께 출력하면 좋을듯!

\$ java Main ../../programs/Factorial.java

Undeclared identifier: num_aux

Undeclared identifier: num_aux

Undeclared identifier: num_aux

Return type mismatch in method ComputeFac:
expected int, found <unknown>

Type check failed for ../../programs/Factorial.java

MiniJava 타입 검사: 실패

In chap5/handcrafted/Main.java

- 12: 변수명 오타 `mum_aux` => `num_aux`

```
7  class Fac {  
8  
9      public int ComputeFac(int num){  
10         int num_aux ;  
11         if (num < 1)  
12             mum_aux = 1 ;  
13         else  
14             num_aux = num * (this.ComputeFac(num-1)) ;  
15         return num_aux ;  
16     }  
17  
18 }
```

\$ java Main ../../programs/Factorial.java
Undeclared identifier: mum_aux

Type check failed for ../../programs/Factorial.java

MiniJava 타입 검사: 실패

In chap5/handcrafted/Main.java

- 12: 타입 오류 Fac num_aux => int num_aux

```
7  class Fac {  
8  
9      public int ComputeFac(int num){  
10         Fac num_aux ;  
11         if (num < 1)  
12             num_aux = 1 ;  
13         else  
14             num_aux = num * (this.ComputeFac(num-1)) ;  
15         return num_aux ;  
16     }  
17  
18 }
```

java Main ../../programs/Factorial.java

Type mismatch in assignment to num_aux:
expected Fac, found int

Type mismatch in assignment to num_aux:
expected Fac, found int

Return type mismatch in method ComputeFac:
expected int, found Fac

Type check failed for ../../programs/Factorial.java

MiniJava 타입 검사: 실패

In chap5/handcrafted/Main.java

- 12: 변수 중복 선언

```
7  class Fac {  
8  
9      public int ComputeFac(int num){  
10         int num_aux ;  
11         int num_aux ;  
12         if (num < 1)  
13             num_aux = 1 ;  
14         else  
15             num_aux = num * (this.ComputeFac(num-1)) ;  
16         return num_aux ;  
17     }  
18  
19 }
```

java Main ../../programs/Factorial.java

...

MiniJava 타입 검사: 실패

In chap5/handcrafted/Main.java

- 12: 메서드 중복 선언

```
7  class Fac {  
8  
9-  public int ComputeFac(int num){  
10     int num_aux ;  
11 >  if (num < 1) ...  
13 >  else ...  
15     return num_aux ;  
16 }  
17  
18 public int ComputeFac(int num) { int x; return x; }  
19 }
```

java Main ../../programs/Factorial.java

...

MiniJava 타입 검사: 실패

In chap5/handcrafted/Main.java

- 20: 클래스 중복 선언

```
7  class Fac {  
8  
9      public int ComputeFac(int num){  
10         int num_aux ;  
11         if (num < 1)  
12             num_aux = 1 ;  
13         else  
14             num_aux = num * (this.ComputeFac(num-1)) ;  
15         return num_aux ;  
16     }  
17  
18 }  
19  
20 class Fac { }
```

java Main ../../programs/Factorial.java

...

MiniJava 타입 검사

MiniJava 타입 검사를 위한 심볼 테이블의 바인딩

- 변수와 형식 매개변수 -> 타입
- 메서드 -> 매개 변수, 반환 타입, 지역 변수
- 클래스 -> 필드 선언, 메서드 선언

예) 그림 5.7 MiniJava 프로그램 예시와 심볼 테이블

MiniJava 타입 검사

```
class B {  
    C f; int [] j; int q;  
    public int start(int p, int q) {  
        int ret; int a;  
        /* ... */  
        return ret;  
    }  
    public boolean stop(int p) {  
        /* ... */  
        return false;  
    }  
}  
  
class C {  
    /* ... */  
}
```

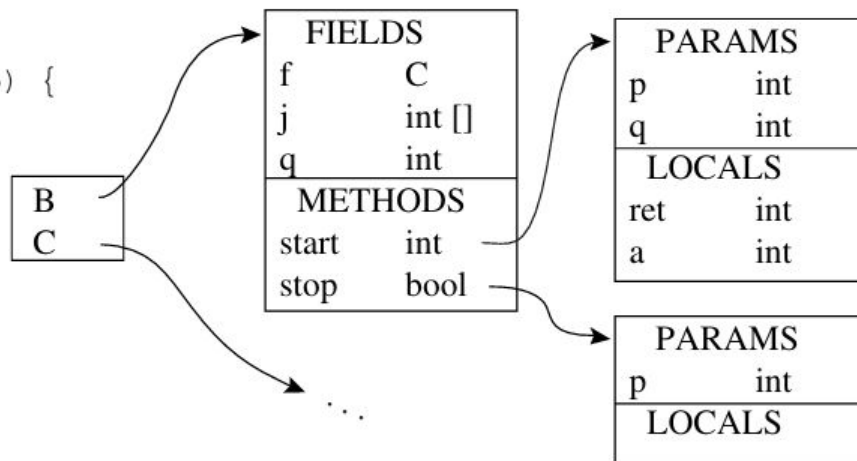


FIGURE 5.7. A MiniJava Program and its symbol table

Chapter 4 syntaxtree AST Class Diagram

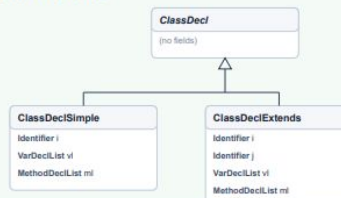
One hollow triangle per parent shows inheritance. Boxes list class name and member variables only.

Special Thaks to
조영진!!

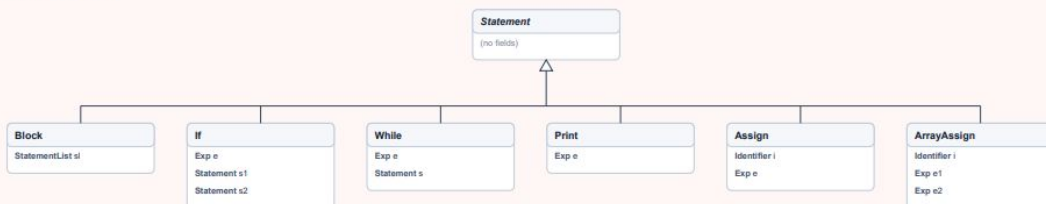
Type inheritance



ClassDecl inheritance



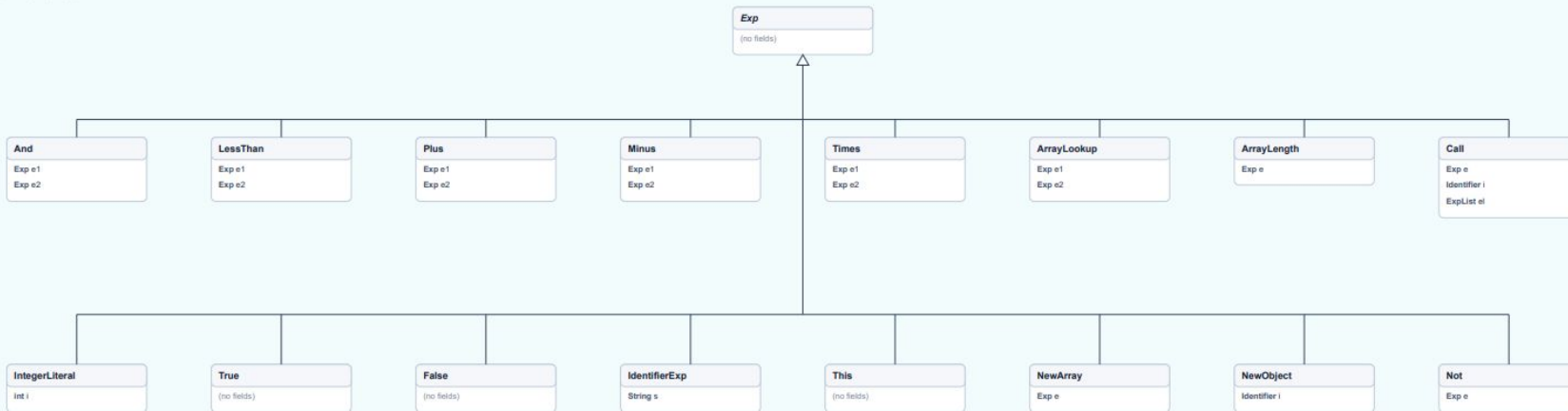
Statement inheritance



Program, declarations, and lists



Exp inheritance



MiniJava 타입 검사

- 클래스와 메서드 정보

```
15 private static class MethodInfo {
16     Type returnType;
17     LinkedHashMap<String, Type> params = new LinkedHashMap<>();
18     HashMap<String, Type> locals = new HashMap<>();
19 }
20
21 private static class ClassInfo {
22     String name;
23     String parent; // may be null
24     HashMap<String, Type> fields = new HashMap<>();
25     HashMap<String, MethodInfo> methods = new HashMap<>();
26 }
27
28 private final HashMap<String, ClassInfo> classes = new HashMap<>();
29 private ClassInfo currentClass = null;
30 private MethodInfo currentMethod = null;
```

MiniJava 타입 검사

두 단계 타입 검사기

- 1단계 : 추상 구문 트리의 클래스, 멤버 변수, 메소드 선언을 순회, 심볼 테이블을 구축
- 2단계 : 추상 구문 트리의 모든 문장과 식을 순회하며 타입 검사 수행(구축한 심볼 테이블 참조)

1,2단계 모두 방문자 패턴으로 구현

MiniJava 타입 검사

- TypeVisitor 인터페이스를 구현한 TypeCheckVisitor 클래스

: COLLECT와 CHECK 단계를 구분하는 열거형 상수와 phase 변수

```
11 public class TypeCheckVisitor implements TypeVisitor {  
12     private enum Phase { COLLECT, CHECK }  
13     private Phase phase = Phase.COLLECT;  
14 }
```


MiniJava 타입 검사

- TypeCheckVisitor.check() 메소드

```
36 public boolean check(Program program) {
37     phase = Phase.COLLECT;
38     program.accept(this);
39     phase = Phase.CHECK;
40     program.accept(this);
41     for (String e : errors) {
42         System.err.println(e);
43     }
44     return errors.isEmpty();
45 }
```

각 `accept()` 메소드는
`visit(Program program)` 메소드를 호출하여
프로그램의 각 요소의 타입을 검사

“AST 노드.`accept()`” ⇒ “AST 노드.`check()`”

- 에러 처리

```
32 private final ArrayList<String> errors = new ArrayList<>();
33
34 public List<String> getErrors() { return errors; }
```

MiniJava 타입 검사

타입 검사기 1단계

- MiniJava 구문 트리를 순회하면서 심볼 테이블을 구축

```
// Type t;  
// Identifier i;  
public void visit(VarDecl n) {  
  
    Type t = n.t.accept(this);  
    String id = n.i.toString();  
  
    if (currMethod == null) {  
        if (!currClass.addVar(id,t))  
            error.complain(id + "is already defined in " + currClass.getId());  
    } else if (!currMethod.addVar(id,t))  
        error.complain(id + "is already defined in "  
            + currClass.getId() + "." + currMethod.getId());  
}
```

MiniJava 타입 검사

타입 검사기 1단계

- MiniJava 구문 트리를 순회하면서 심볼 테이블을 구축
- Program 5.8의 `visit` 메서드는 변수 선언에서 심볼 테이블 정보를 구축
(변수 선언 처리) 변수의 이름과 타입을 현재 클래스의 자료 구조에 추가
(중복 선언 검사) 같은 변수가 중복 선언되었는지 검사 => 오류 메시지 출력

MiniJava 타입 검사

타입 검사기 2단계

- 1단계에서 구축한 심볼 테이블을 참조하여 모든 문장과 식의 타입을 검사

각 `accept()` 메소드는
`visit(Exp exp)` 메소드를 호출하여
덧셈의 두 연산자 노드의 타입을 검사

`n.e1.accept()` $\Rightarrow$ `n.e1.check()`
`n.e2.accept()` $\Rightarrow$ `n.e2.check()`

```
// Exp e1,e2;
public Type visit(Plus n) {
    if (! (n.e1.accept(this) instanceof IntegerType) )
        error.complain("Left side of LessThan must be of type integer");
    if (! (n.e2.accept(this) instanceof IntegerType) )
        error.complain("Right side of LessThan must be of type integer");
    return new IntegerType();
}
```

PROGRAM 5.9. A visit method for plus expressions

MiniJava 타입 검사

타입 검사기 2단계

- 1단계에서 구축한 심볼 테이블을 참조하여 모든 문장과 식의 타입을 검사
- 각 **visit** 메소드의 반환 타입은 (MiniJava의 타입을 표현하는) **String**
- 방문자가 어떤 식을 방문하면, 그 식의 타입을 반환
- 그 식이 타입 규칙을 만족하지 않으면, 오류 메시지를 내고 타입 검사를 종료

MiniJava 타입 검사

MiniJava 타입 검사 명세

- JensPalsberg/miniJava-typesystem.pdf

The MiniJava Type System

Jens Palsberg

October 2014

1 What is MiniJava?

MiniJava is a subset of Java. The meaning of a MiniJava program is given by its meaning as a Java program. Overloading is not allowed in MiniJava. The MiniJava statement `System.out.println(...);` can only print integers. The MiniJava expression `e.length` only applies to expressions of type `int[]`.

We will now specify MiniJava's syntax and type system. A MiniJava program will type check with the MiniJava type system (as specified below) if and only if it will type check with the Java type system (as specified in the Java Language Specification).

MiniJava 타입 검사

덧셈식 $e1 + e2$

- 두 피연산자가 모두 정수형이어야 함
- 이 조건이 만족되면, 덧셈식의 결과 타입은 정수형

7.7 Expressions and Primary Expressions

$$\frac{A, C \vdash p_1 : \text{boolean} \quad A, C \vdash p_2 : \text{boolean}}{A, C \vdash p_1 \ \&\& \ p_2 : \text{boolean}} \quad (32)$$

$$\frac{A, C \vdash p_1 : \text{int} \quad A, C \vdash p_2 : \text{int}}{A, C \vdash p_1 < p_2 : \text{boolean}} \quad (33)$$

$$\frac{A, C \vdash p_1 : \text{int} \quad A, C \vdash p_2 : \text{int}}{A, C \vdash p_1 + p_2 : \text{int}} \quad (34)$$

명세: 타입 환경 A 와 현재 클래스 C 에서 p_1 의 타입이 int 이고, p_2 의 타입이 int 이면
동일한 A, C 에서 $p_1 + p_2$ 식의 타입이 int

$$\frac{A, C \vdash p_1 : \text{int} \quad A, C \vdash p_2 : \text{int}}{A, C \vdash p_1 + p_2 : \text{int}}$$

구현: $\text{typeCheck}(A, C, p_1 + p_2) \Rightarrow \text{int}$

Java 구현: $\text{typeCheck}(A, C, p_1 + p_2) \Rightarrow \text{int}$

```
public Type visit(Plus n) {  
    if (phase == Phase.COLLECT) return null;  
    Type a = n.e1.accept(this);  
    Type b = n.e2.accept(this);  
    if (!(a instanceof IntegerType) || !(b instanceof IntegerType)) {  
        error("Addition requires int operands, found " + typeName(a) + " and " + typeName(b));  
    }  
    return new IntegerType();  
}
```



MiniJava 타입 검사

대입문(assignment) $x = e$

- 왼쪽 x 의 타입과 오른쪽 e 의 타입이 같은지 검사
- 클래스 상속을 허용하는 경우에는 이 조건이 조금 완화됨
: 오른쪽 식의 타입이 왼쪽 타입의 서브타입(subtype)

7.6 Statements

$$\frac{A, C \vdash s_i \quad i \in 1..q}{A, C \vdash \{ s_1 \dots s_q \}} \quad (26)$$

$$\frac{A(id) = t_1 \quad A, C \vdash e : t_2 \quad t_2 \leq t_1}{A, C \vdash id = e;} \quad (27)$$

MiniJava 타입 검사

대입문(assignment) $x = e$

$$\frac{A, C \vdash s_i \quad i \in 1..q}{A, C \vdash \{ s_1 \dots s_q \}} \quad (26)$$

$$\frac{A(id) = t_1 \quad A, C \vdash e : t_2 \quad t_2 \leq t_1}{A, C \vdash id = e;} \quad (27)$$

```
public Type visit(Assign n) {
    if (phase == Phase.COLLECT) return null;
    Type lhs = lookupVar(n.i.s);
    if (lhs == null) {
        error("Undeclared identifier: " + n.i.s);
        return null;
    }
    Type rhs = n.e.accept(this);
    if (!(sameType(rhs, lhs) || isSubtype(rhs, lhs))) {
        error("Type mismatch in assignment to " + n.i.s +
            ": expected " + typeName(lhs) + ", found " + typeName(rhs));
    }
    return null;
}
```

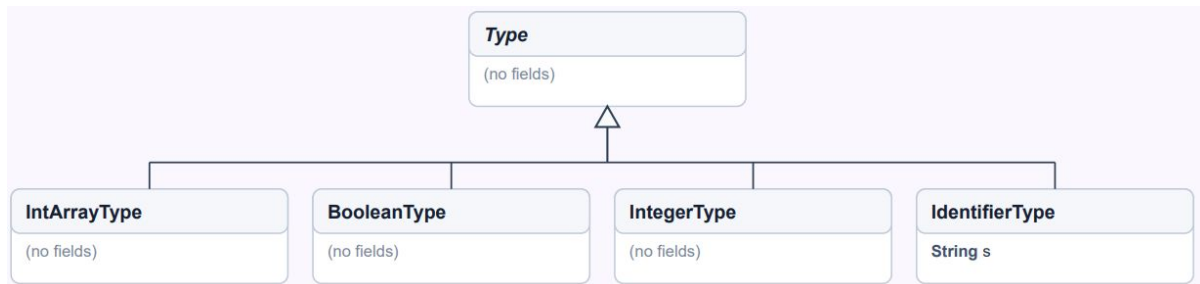
Assign

Identifier i

Exp e

MiniJava 타입 검사

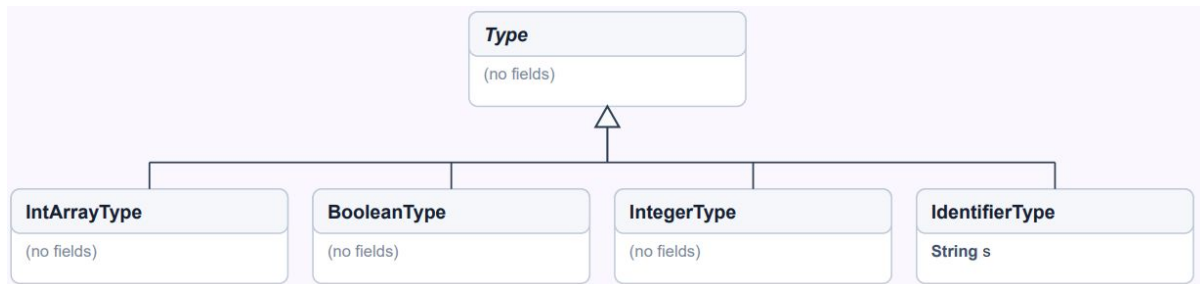
sameType



```
private boolean sameType(Type a, Type b) {
    if (a == null || b == null) return false;
    if (a.getClass() == b.getClass()) {
        if (a instanceof IdentifierType) {
            String sa = ((IdentifierType)a).s;
            String sb = ((IdentifierType)b).s;
            return sa.equals(sb);
        }
        return true;
    }
    return false;
}
```

MiniJava 타입 검사

isSubtype



```
private boolean isSubtype(Type sub, Type sup) {
    if (sub == null || sup == null) return false;
    if (sameType(sub, sup)) return true;
    if (!(sub instanceof IdentifierType) || !(sup instanceof IdentifierType)) return false;
    String s = ((IdentifierType)sub).s;
    String t = ((IdentifierType)sup).s;
    // climb extends chain
    ClassInfo ci = classes.get(s);
    while (ci != null && ci.parent != null) {
        if (t.equals(ci.parent)) return true;
        ci = classes.get(ci.parent);
    }
    return false;
}
```

MiniJava 타입 검사

메서드 호출 $p.id(e_1, \dots, e_k)$

```
public class TypeCheckVisitor implements TypeVisitor {  
    private enum Phase { COLLECT, CHECK }  
    private Phase phase = Phase.COLLECT;  
  
    private static class MethodInfo {  
        Type returnType;  
        LinkedHashMap<String, Type> params = new LinkedHashMap<>();  
        HashMap<String, Type> locals = new HashMap<>();  
    }  
}
```

- 메서드 이름 id 를 심볼 테이블에서 찾아 매개변수 목록과 반환 타입을 얻음
- p 의 타입이 클래스 타입 D 라면, 이 클래스 D 에서 메서드 id 의 정의를 찾음
- 메서드의 매개변수 타입들이 호출식에 주어진 인자들과 일치하는지 검사
- 그 메서드의 반환 타입이 전체 메서드 호출식의 타입이 됨

7.7 Expressions and Primary Expressions

$$\frac{\begin{array}{l} A, C \vdash p : D \quad \text{methodtype}(D, id) = (t'_1, \dots, t'_n) \rightarrow t \\ A, C \vdash e_i : t_i \quad t_i \leq t'_i \quad i \in 1..n \end{array}}{A, C \vdash p.id(e_1, \dots, e_n) : t} \quad (39)$$

MiniJava 타입 검사

모든 종류의 문장과 식은 각각의 고유한 타입 검사 규칙을 갖음

- 여기서 아직 설명하지 않은 나머지 경우의 규칙들은 **Java Language Specification**을 참고하여 도출할 수 있음

(<https://docs.oracle.com/javase/specs/jls/se21/html/>)

[실습] MiniJava 타입 검사기

MiniJava 프로그램을 타입 검사하고, 타입 불일치나 선언되지 않은 식별자에 대해 적절한 오류 메시지를 출력하는 방문자들 설계하시오.